Score Ranked Retrieval Methods for Semi-Structured Knowledge Bases: Addressing Multi-Hop Question Answering Through Joint Structural and Semantic Reasoning

Amit Joshi[1][0000-1111-2222-3333], Aditya Deshmukh[1][1111-2222-3333-4444], Anshul Shelokar[1][2222–3333-4444-5555], Payal Sankpal[1][2222–3333-4444-5555], and Sarthak Pawar[1][2222–3333-4444-5555]

COEP Technological University, Pune, India
Department of Computer Engineering
email@coeptech.ac.in

Abstract. Large Language Models (LLMs) have demonstrated remarkable capabilities in natural language understanding, yet they suffer from hallucinations when generating responses without grounding in factual knowledge. Retrieval Augmented Generation (RAG) addresses this by incorporating external knowledge sources. However, retrieval on semi-structured knowledge bases (SKBs) that contain both textual information and relational structure remains challenging. Existing methods rely heavily on LLMs to extract precise structural constraints in a single step, making them vulnerable to hallucinations leading to inaccurate responses. These errors cascade through the retrieval pipeline, leading to poor performance at downstream retrieval . We propose a novel hybrid retrieval framework that addresses these limitations through two key innovations: (1) flexible, multi-stage LLM inferences that accept multiple possible values than precise single values, and (2) a priority queue- based grounding algorithm that dynamically expands the candidate set rather than filtering it based on potentially incorrect LLM outputs. Our approach uses LLM outputs as a broad set of signals for searching rather than hard constraints, enabling the system to recover from LLM errors.

Keywords: Semi-Structured Knowledge Bases · Retrieval Augmented Generation · Multi-hop Question Answering · Hybrid Retrieval

1 Introduction

Large Language Models have rapidly evolved in recent years, demonstrating strong language understanding and instruction following capabilities [1] in complex tasks. However, despite the remarkable generative powers of LLMs, they suffer from the problem of hallucinations [2], when LLMs generate false information that appears to satisfy the query. This limitation becomes particularly

problematic in knowledge-intensive applications where accuracy is paramount such as biomedical question answering. To mitigate this problem, LLM responses were augmented with external up-to-date knowledge sources in RAG - Retrieval Augmented Generation [3]. LLMs use the query to retrieve relevant additional information from external sources and use both the query and the additional information to generate answers, which improves the factual accuracy of responses [4]. These external sources can be structured like knowledge graphs[5–8] or relational tables[9, 10], or unstructured data like natural language documents in the form of vector databases [11, 12].

The evolution of retrieval mechanisms has progressed from the initial focus on passage retrieval from unstructured free-text document collections [13–15] to addressing increasingly complex and heterogeneous data structures. A particularly significant development has been the emergence of text-rich graph knowledge bases (TG-KBs), also referred to as Semi-Structured Knowledge Bases (SKBs).

SKBs present a powerful way to store data that is inherently hybrid in nature. For example, academics retrieve relevant papers from systems where nodes represent papers and edges represent references between them. SKBs organize: (i) **relational** (structural) information in the form of edges between nodes, and (ii) **textual** information in the form of text documents attached to each node. The challenge of retrieving relevant nodes from such graphs requires a unified framework that combines both modalities - structural connectivity and textual semantic similarity.

Traditional retrieval approaches focus either on structural constraints or on textual similarity [16–18], failing to address all constraints provided by the query. Recent work has explored specialized techniques for semi-structured retrieval, including pseudo-relevance feedback approaches [19] and multi-field adaptive retrieval [20], which demonstrate the necessity of field-aware and context-adaptive strategies. Additionally, knowledge-aware query expansion with LLMs has shown promise in handling semi-structured queries with both textual and relational requirements [21]. Effective retrieval in SKBs requires the development of hybrid retrieval methods that can jointly reason over graph topology and relational information, as well as rich textual descriptions associated with graph nodes, enabling the system to handle all query requirements and perform complex multi-hop question answering.

Recent methods have made substantial progress in this direction. Agent-based approaches leverage planning and reasoning to traverse semi-structured data [22], while hybrid retrieval-augmented generation systems combine both textual and graph-based retrievers [23]. Knowledge graph-integrated retrieval frameworks enhance LLM reasoning with structured knowledge [24], and more sophisticated methods employ mixture-of-experts strategies to dynamically balance structural and textual knowledge retrieval [25]. Graph database-native approaches with fine-tuned LLMs demonstrate the value of generating provably correct graph queries for high-quality context retrieval [26]. Furthermore, specialized frameworks focused on semi-structured knowledge bases have achieved

state-of-the-art performance through modular integration of entity search, query generation, and ranking components [27].

This proposed framework addresses these challenges by developing a unified approach that effectively combines structural and semantic reasoning for retrieval in semi-structured knowledge bases. Our approach leverages the complementary strengths of both knowledge modalities: textual similarity enables semantic understanding of content while structural knowledge provides logical constraints and relational context. By integrating these dimensions through a cohesive retrieval pipeline, we enable more precise multi-hop reasoning and higher-quality answer generation for complex queries over heterogeneous semi-structured data.

2 Related Work

2.1 Unstructured and Structured Document Retrieval

The task of document retrieval addresses a broad range of document types, ranging from textual document retrieval to structured entity retrieval from knowledge graphs. Textual document retrieval, particularly the retrieval of top-k relevant documents given a natural language query, has been studied extensively. BM25 [28] based on lexical matching serve as baselines for this task, but they fail to understand the underlying semantics of queries and documents in the corpus, leading to sub-optimal results. This gap was addressed by dense retrieval models such as DPR [16], ColBERT/ColBERTv2,[17, 18] Contriever[29], and ANCE [30], which convert both queries and documents to low-dimensional dense vector embeddings that preserve semantic context, leading to better results where simple term-based lexical matching fails.Vector Similarity Search (VSS) was used to address the semantic understanding challenge using embedding models like text-embedding-ada-002 (abbrev. ada-002) [31], voyage- large-2-instruct (abbrev. voyage-l2-instruct) [32], LLM2Vec-Meta-Llama-3-8B-Instruct-mntp (abbrev. LLM2Vec) [33], and GritLM-7b[34].

Unstructured documents, however, are not the most efficient way of storing information, as many modern applications deal with data that have inherent structure and are relational in nature. Knowledge graphs are used to store such structured relational information wherein entities are represented by nodes in graphs, and relations between entities are denoted by edges. These graphs can be heterogeneous with different node types and relation types. Graph-based RAG methods such as QA-GNN [35], Think-on-Graph [36], and others [37] have been explored to retrieve the most relevant nodes from structured knowledge bases given natural language queries, the results of which are then used to perform downstream tasks like providing additional context to LLMs in RAG pipelines. These structured knowledge bases handle the relational aspects of data but lack rich textual/semantic information about node entities, limiting their expressive power.

2.2 Semi-Structured Knowledge Base Retrieval

Semi-structured knowledge bases address the limitations of structured and unstructured knowledge representation by representing relational information using graph edges between nodes and associating rich textual descriptions to each node for storing additional semantic information about entities in the database. Retrieval on such SKBs becomes a challenging task, as it requires considering both structural/relational information and semantic constraints.

Several frameworks have been proposed for hybrid retrieval on semi-structured knowledge graphs. In recent frameworks, LLMs have been used extensively in multiple stages of the retrieval task, including generating retrieval paths from graphs, re-ranking, and other stages. However, despite their generative capabilities and understanding of language semantics, LLMs still struggle to deal with the mixture of structural and relational aspects of semi-structured retrieval tasks. The STARK benchmark [38] has been proposed to test the performance of hybrid retrieval on semi-structured knowledge bases having natural language queries that require considering both structural and semantic information to reason over multiple entities (multi-hop) to retrieve the most relevant answer nodes.

Some of the proposed frameworks include:

AVATAR [39] is an agent-based approach that utilizes a comparator module, generating contrastive prompts to optimize LLM behavior during tool usage.

HybGRAG [23] uses a retrieval bank with an LLM router to analyze queries and find topic entities and ego graphs, then uses a retrieval bank that supports both hybrid and pure semantic retrieval as a fallback to collect top-k relevant documents. These documents are then fed to a validator and critic agent, which checks the validity of results and provides specific feedback to the router module to iteratively improve routing.

mFAR [24] uses both lexical and semantic similarity for each field in the data. They also introduce adaptive weighting between these lexical and semantic match scores on each field, conditioning on the query.

PASemiQA [22] is another agentic approach that first generates a plan including matching entities and relation paths. This plan is then fed to an LLM-based agent traversal module, which follows a thought and action iterative repetitive process to explore SKBs and extract relevant nodes.

FocusedRetriever [27] (CYPHER) uses LLMs to generate cypher query from the given natural language query, then parses the cypher query to extract symbols, triplets and constraints and then finds and grounds candidates for each symbol to arrive at a set of possible answer nodes.

GraphRAFT[26] (CYPHER) finetunes an LLM to generate cypher queries that are directly executed on a graph database to retrieve relevant nodes.

MoR[25] organizes retrieval into 3 parts: planning, reasoning, and organizing. A fine-tuned LLM is used to generate planning graphs from the query. This plan is used to perform step-by-step reasoning on the SKB, and finally, the retrieved nodes are re-ranked using their structural trajectory.

KAR[40] initially identifies entities and retrieves a set of initial triplets, passing them as context to the LLM for knowledge aware query expansion. The expanded query is then used for downstream knowledge base retrieval.

3 Problem Statement

We provide a formal definition of the problem as follows:

Given a semi-structured knowledge graph $G(V, E, D)$ where:

- V is the set of nodes,
- $E \subseteq V \times V$ is the set of edges representing relationships between nodes, and
- $D = \bigcup_{i \in V} D_i$ is the collection of free-form text documents associated with the nodes, where D_i is the set of documents associated with node i.

We define the task of retrieval on a semi-structured knowledge base as follows:

Given the knowledge graph $G = (V, E, D)$, a collection of free text documents D, and a query Q, the **output is a set of nodes** $A \subseteq V$ such that for each node $i \in A$, it satisfies the relational requirements imposed by the structure of G as specified in Q, and the associated documents D_i satisfy the textual requirements specified in Q.

4 Proposed Methodology

4.1 Overview and System Architecture

Our proposed framework consists of six main components that process natural language queries to retrieve relevant entities from a Structured Knowledge Base (SKB). The overall workflow proceeds as follows:

1. **Entity Identification:** An LLM analyzes the input query to extract mentioned entities and identify constraints (both lexical and semantic) associated with each entity. Importantly, the LLM outputs a list of possible entity types for each symbol rather than committing to a single type.
2. **Relation Identification:** Given the extracted entities, a separate LLM prompt identifies relationships between entity pairs implied by the query, again producing lists of possible relation types.
3. **Initial Candidate Retrieval:** For each non-answer entity symbol, we retrieve an initial set of candidate nodes from the SKB using a combination of exact lexical matching and vector similarity search.
4. **Priority Queue-Based Grounding:** We initialize priority queues for each symbol with initial candidates. The core grounding algorithm iteratively pops candidates, explores their graph neighborhoods based on identified relation types, and adds newly discovered nodes to relevant priority queues. A support boost mechanism increases scores for candidates that satisfy multiple relational constraints.

5. **Ranking:** After collecting candidates for the answer node, we compute final scores based on semantic similarity and structural support.
6. **LLM Reranking:** The top-ranked candidates along with supporting context (triplets, related entities) are passed to an LLM for final reranking to produce the ultimate result list.

This architecture deliberately separates entity extraction, relation extraction, and grounding into distinct stages, with each stage designed to be robust to imperfect inputs from previous stages. The key innovation lies in the flexible entity/relation typing and the expansive grounding strategy that treats constraints as suggestions rather than requirements.

4.2 Entity Identification Module

Design Principles The entity identification stage aims to extract all entities mentioned in the query and characterize them along three dimensions: entity type(s), lexical constraints (explicit names or identifiers), and semantic constraints (descriptive properties or relationships expressed in natural language). Unlike prior work that requires the LLM to output a single entity type per entity, we allow multiple possible types to accommodate ambiguity and reduce the brittleness of downstream processing. The prompt templates are provided in Appendix.

Multi-Type Entity Mapping A critical aspect of our approach is allowing entities to map to multiple types. For example, "hypertension" could be classified as both a disease and an effect/phenotype, as it represents a medical condition that can also manifest as a symptom or outcome of other conditions. By accepting multiple types from the LLM, we avoid forcing premature disambiguation and enable the grounding algorithm to explore candidates of various types.

Example Output For the query "Find medical conditions linked to the gene SLC29A3 that should not be treated with the drug Goserelin," the entity identification module produces:

The `constant` field indicates whether the entity is explicitly mentioned by name (A and B) or is an unknown entity to be retrieved (ANSWER). Semantic constraints for the answer node capture the relational requirements expressed in the query.

4.3 Relation Identification Module

Design Principles The relation identification stage takes the entity symbols from the previous stage and determines how they are connected in the SKB's graph structure. As with entity types, we allow the LLM to propose multiple possible relation types for each entity pair, acknowledging that natural language often underdetermines the precise formal relationship.

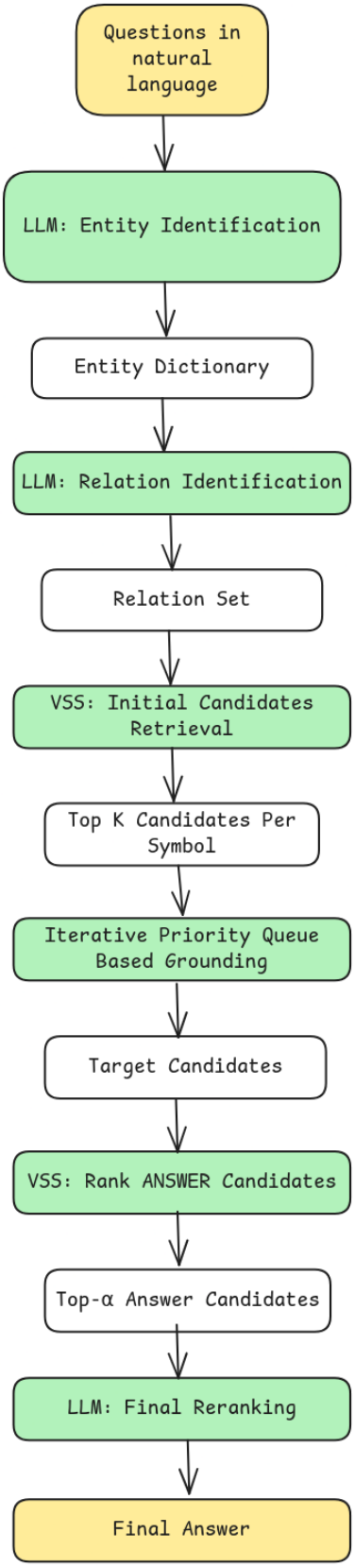

Fig. 1. System architecture

```
{
  'A': {
    'type': ['gene/protein'],
    'lexical': {'name': 'SLC29A3'},
    'semantic': [],
    'constant': True
  },
  'B': {
    'type': ['drug'],
    'lexical': {'name': 'Goserelin'},
    'semantic': [],
    'constant': True
  },
  'ANSWER': {
    'type': ['disease'],
    'lexical': {},
    'semantic': [
      'linked to the gene SLC29A3',
      'should not be treated with the drug Goserelin'
    ],
    'constant': False
  }
}
```

Fig. 2. Output of Entity identification module

Constraint on Answer Node We enforce that there must be at least one relation involving the ANSWER entity, as the answer must be connected to at least one known entity in the query. This ensures that the grounding algorithm has a starting point for discovering answer candidates through graph traversal.

Example Output Continuing the previous example:

```
{
  ('A', 'ANSWER'): ['associated_with'],
  ('B', 'ANSWER'): ['contraindication']
}
```

Fig. 3. Output of Relation identification module

This indicates that the answer should be associated with entity A (gene SLC29A3) and have a contraindication relationship with entity B (drug Gosere-

lin). The specific relation type "contraindication" is inferred from the phrase "should not be treated with."

4.4 Initial Candidate Retrieval

Exact Matching For entities with explicit lexical names, we first perform exact string matching against node names in the SKB. If an exact match is found, that node is added to the candidate set with a confidence score of 1.0, indicating high certainty. This ensures that explicitly mentioned entities are reliably grounded to their corresponding nodes.

Vector Similarity Search After exact matching, we use dense embeddings to find additional candidates based on semantic similarity. We construct a query representation by concatenating the entity's type(s), lexical name (if present), and semantic descriptions. This composite description is encoded using a pre-trained embedding model (e.g., text-embedding-ada-002). We then perform vector similarity search in the embedding space to retrieve the top-n (typically 20-25) most similar nodes from the SKB, filtering by entity type(s) predicted by the LLM. The similarity score (typically cosine similarity) serves as the initial priority for these candidates.

Handling the Answer Entity For the ANSWER entity, which has no lexical name, we rely entirely on semantic descriptions. However, we do not retrieve initial candidates for the answer node. Instead, answer candidates are discovered through the grounding process by exploring neighborhoods of known entities. This approach ensures that answer candidates naturally satisfy at least some structural constraints.

4.5 Priority Queue-Based Grounding Algorithm

Algorithm Overview The grounding algorithm represents the core contribution of our method. It maintains a priority queue for each entity symbol, where queue elements are candidate nodes with associated scores. The algorithm proceeds in rounds, processing candidates in a round-robin fashion across all non-answer symbols. In each iteration, we pop the highest-priority candidate from a symbol's queue, explore its graph neighborhood according to identified relations, and add newly discovered nodes to the appropriate queues.

Priority Queue Initialization For each entity symbol S (excluding ANSWER), we initialize its priority queue Q_s with the initial candidates retrieved in Section 4.5. Each queue entry is a tuple $(node\_id, score, context)$ where:

- **node_id:** The identifier of the candidate node in the SKB.
- **score:** The initial similarity score from vector search (or 1.0 for exact matches).

– **context:** A data structure tracking supporting evidence for this candidate, including the list of triplets that connect it to other candidates and references to candidates in other symbols.

The priority queue is implemented as a max-heap, ensuring that the candidate with the highest score is always at the front.

Round-Robin Traversal Instead of exhaustively processing all candidates from one symbol before moving to the next, we employ a round-robin strategy that alternates between symbols. In each round, we process one candidate from each non-answer symbol's queue. This approach provides two benefits:

1. **Balanced Exploration:** It prevents the algorithm from getting stuck exploring a single branch of the graph while neglecting other symbols.
2. **Cross-Entity Synergy:** Candidates for different symbols can be discovered and reinforced simultaneously, accelerating convergence.

The round-robin continues until we have accumulated a sufficient number of answer candidates (typically 50-100) or until all queues are exhausted.

Neighbor Exploration and Filtering When processing a candidate c for symbol S, we examine all edges incident to c in the SKB. For each relation type R in the identified relation list for (S, T) (where T is another symbol), we collect neighbors of c that are connected via an edge of type R. If the resulting neighbor set is large (more than a threshold k, typically 20), we apply semantic filtering to retain the top-k most relevant neighbors. Relevance is computed as the cosine similarity between the neighbor's embedding and the target symbol T's semantic description.

Dynamic Candidate Addition For each filtered neighbor n, we check if n is already present in the target symbol's candidate set. If n is new, we add it to the priority queue Q_T with a score computed as:

$$score(n) = \alpha \cdot sim(n, T) \cdot decay \tag{1}$$

where $sim(n, T)$ is the semantic similarity between n and symbol T, decay is a decay factor (typically 0.9) that slightly reduces scores for candidates discovered through traversal rather than direct retrieval, and α is a weighting factor.

5 Experiments

5.1 Experimental Setup

We use off the shelf LLaMa 3.3 70B as our LLM backbone, with the temperature parameter set to 0.2 to control the randomness and allow for little creativity in responses.

We use text-embedding-ada-002 for generating dense vector embeddings of documents and queries. This model demonstrates state-of-the-art performance on retrieval and semantic similarity tasks due to larger embedding space and strong semantic embedding capability.

We perform LLM reranking using the same LLM, LLaMa 3.3 70B, on the top 20 retrieved documents to allow a fair comparision with exisiting frameworks that have a reranking step. We cap the maximum number of candidates considered per symbol to 3000.

We set the hyperparamters as follows, based on performance on validation set: initial candidates per symbol as 25, maximum candidates considered per symbol while grounding to 3000, a score decay factor of 0.9 and support boost of 0.35.

5.2 Dataset

We test our methodology on all three of the datasets proposed in the STaRK benchmark : AMAZON, MAG, and PRIME. Table 1 shows information about each of these semi-structured knowledge bases (SKBs), namely entity type counts, relation type counts, node and edge quantities, mean node degree, and overall token numbers.

The STaRK benchmark contains a collection of synthetically created queries divided into train, validation, and test splits, along with a smaller collection of human generated queries. The queries replicate real world conditions by combining relational and textual information phrased in natural sounding and varied ways. Following conventions used when reporting results for other large language model approaches, we report the evaluation results on 10% of the synthetic test queries, on a subset specified by the STaRK benchmark.

The three SKBs exhibit notable differences:

- **PRIME** (healthcare) contains qeuries with higher complexity and are highly domain specific in nature, with varied node and edge types with higher emphasis on relational information.
- **AMAZON** (e-commerce) emphasises more on rich textual information such as product descriptions and customer reviews, while incorporating some structural and relational information.
- **MAG** (academic literature) strikes balance between relational requirements such as citation and authorship relations and textual requirements such as titles and abstracts.

Each SKB contains between hundreds of thousands and millions of nodes and edges. Within PRIME, any node type can be a valid answer. For MAG and AMAZON, only certain node categories, specifically papers and products respectively, can be valid answers.

5.3 Performance Metrics

We report average scores over test queries for the following performance metrics:

Table 1. Data Statistics for datasets from the STaRK Benchmark

Dataset	#Node Types	#Edge Types	Avg. Degree	#Nodes	#Edges	#Queries	Train/Val/Test
Amazon	4	4	18.2	1,035,542	9,443,802	9,100	0.65/0.17/0.18
MAG	4	4	43.5	1,872,968	39,802,116	13,323	0.60/0.20/0.20
PrimeKG	10	18	125.2	129,375	8,100,498	11,204	0.55/0.20/0.25

Table 2. Performance comparison across benchmark methods and method on synthetically generated datasets

Dataset	AMAZON				MAG				PRIME				Average			
Metric	H1	H5	R20	MRR	H1	H5	R20	MRR	H1	H5	R20	MRR	H1	H5	R20	MRR
BM25	44.9	67.4	53.8	55.3	25.8	45.2	45.7	34.9	12.8	27.9	31.2	19.8	27.8	46.9	43.6	36.7
VSS (Ada-002)	39.2	62.7	53.3	50.4	29.1	49.6	48.4	38.6	12.6	31.5	36.0	21.4	27.0	47.9	45.9	36.8
VSS (Multi-Ada)	40.1	65.0	55.1	51.6	25.9	50.4	50.8	36.9	15.1	33.6	38.0	23.5	27.0	49.7	48.0	37.3
DPR	15.3	47.9	44.5	30.2	10.5	35.2	42.1	21.3	4.5	21.8	30.1	12.4	10.1	35.0	38.9	21.3
VSS + Reranker	44.8	71.2	55.4	55.7	40.9	58.2	48.6	49.0	18.3	37.3	34.1	26.6	34.7	55.6	46.0	43.8
VSS + Reranker	45.5	71.1	53.8	55.9	36.5	53.2	48.4	44.2	17.8	36.9	35.6	26.3	33.3	53.7	45.9	42.1
GTA-GNN	26.6	50.0	52.0	37.8	12.9	39.0	47.0	29.1	8.8	21.4	29.6	14.7	16.1	36.8	42.9	27.2
ToG	-	-	-	-	13.2	16.2	11.3	14.2	6.1	15.7	13.1	10.2	-	-	-	-
AvaTaR	49.9	69.2	**60.6**	58.7	44.4	59.7	50.6	51.2	18.4	36.7	39.3	26.7	37.6	55.2	50.2	45.5
4StepFocus	47.6	67.6	56.2	56.5	53.8	69.2	65.8	61.4	39.3	53.2	55.9	45.8	46.9	63.3	59.3	54.6
KAR	*54.2*	68.7	57.2	61.3	50.5	69.6	60.3	58.6	30.4	49.3	50.8	39.2	45.0	62.5	56.1	53.0
HybGRAG	-	-	-	-	*65.4*	*75.3*	65.7	*69.8*	28.6	41.4	43.6	34.5	-	-	-	-
ReAct	42.1	64.6	50.8	52.3	31.1	49.5	47.0	39.2	15.3	32.0	33.6	22.8	29.5	48.7	43.8	38.1
Reflexion	42.8	65.0	54.7	52.9	40.7	54.4	49.6	47.1	14.3	35.0	38.5	24.8	32.6	51.5	47.6	41.6
PASemiQA	45.9	-	-	55.7	43.2	-	-	50.2	29.7	-	-	31.0	39.6	-	-	45.6
mFAR	41.2	70.0	58.5	54.2	49.0	69.6	71.7	58.2	40.9	62.8	**68.3**	51.2	43.7	67.5	66.2	54.5
MoR	52.2	*74.6*	59.9	62.2	58.2	78.3	*75.0*	67.1	36.4	60.0	63.5	46.9	48.9	-	-	58.8
FocusedR.	**64.0**	**76.2**	56.0	**69.3**	**74.1**	**84.2**	**78.8**	**78.8**	**46.4**	**63.9**	65.5	**53.7**	**61.5**	**74.8**	**66.7**	**67.3**
ScoreRankedR.	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	**0.0**	**0.0**

- **Hit@k:** 1 if at least one ground truth appears in the top-k results.
- **Recall@k:** Ratio of ground truths present in the top-k results.
- **Mean Reciprocal Rank (MRR):** Reciprocal of the rank of the first ground truth in the results, limited to the top-k results.

5.4 Results and Discussion

[Results and analysis would be presented here]

6 Conclusion

[Conclusion and future work would be discussed here]

References

1. Wei, J., et al.: Finetuned Language Models are Zero-Shot Learners. In: International Conference on Learning Representations (2022)

2. Ji, Z., et al.: Survey of Hallucination in Natural Language Generation. ACM Computing Surveys 55(12), Article 248 (2023)
3. Agrawal, G., Kumarage, T., Alghamdi, Z., Liu, H.: Can knowledge graphs reduce hallucinations in llms?: A survey. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, vol. 1, pp. 3947–3960 (2024)
4. Zhao, P., et al.: Retrieval-Augmented Generation for AI-Generated Content: A Survey. arXiv preprint arXiv:2402.19473 (2024)
5. Akari Asai, Kazuma Hashimoto, Hannaneh Hajishirzi, Richard Socher, and Caiming Xiong. 2020. Learning to Retrieve Reasoning Paths over Wikipedia Graph for Question Answering. In ICLR.
6. Jonathan Berant, Andrew Chou, Roy Frostig, and Percy Liang. 2013. Semantic Parsing on Freebase from Question-Answer Pairs. In EMNLP
7. Shulin Cao, Jiaxin Shi, Liangming Pan, Lunyiu Nie, Yutong Xiang, Lei Hou, Juanzi Li, Bin He, and Hanwang Zhang. 2022. KQA Pro: A Dataset with Explicit Compositional Programs for Complex Question Answering over Knowledge Base. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers).
8. Tiantian Gao, Paul Fodor, and Michael Kifer. 2019. Querying Knowledge via Multi-Hop English Questions. Theory and Practice of Logic Programming (2019).
9. Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. Association for Computational Linguistics, Brussels, Belgium.
10. Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning. CoRR abs/1709.00103 (2017)
11. Matthew Dunn, Levent Sagun, Mike Higgins, V. Ugur Guney, Volkan Cirik, and Kyunghyun Cho. 2017. SearchQA: A New QA Dataset Augmented with Context from a Search Engine. arXiv:1704.05179 [cs.CL]
12. Yao, S., et al.: ReAct: Synergizing Reasoning and Acting in Language Models. In: The Eleventh International Conference on Learning Representations (2023)
13. Bonifacio, L., et al.: InPars: Data Augmentation for Information Retrieval Using Large Language Models. arXiv preprint arXiv:2202.05144 (2022)
14. Dai, Z., et al.: Promptagator: Few-shot Dense Retrieval From 8 Examples. arXiv preprint arXiv:2209.11755 (2022)
15. Zero-shot Neural Passage Retrieval via Domain-targeted Synthetic Question Generation. In: Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics, pp. 1075–1088 (2021)
16. Karpukhin, V., et al.: Dense passage retrieval for open-domain question answering. In: EMNLP, vol. 1, pp. 6769–6781 (2020)
17. Khattab, Omar, and Matei Zaharia. "Colbert: Efficient and effective passage search via contextualized late interaction over bert." Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval. 2020.
18. Santhanam, Keshav, et al. "Colbertv2: Effective and efficient retrieval via lightweight late interaction." arXiv preprint (2021).
19. Chen, Z., et al.: Pseudo-Relevance Feedback Method Based on the Topic Relevance Model. In: Proceedings of the Information Retrieval Conference (2022)

20. Li, M., Chen, T., Van Durme, B., Xia, P.: Multi-Field Adaptive Retrieval. In: Findings of the Association for Computational Linguistics (2024)
21. Xia, Y., Wu, J., Kim, S., Yu, T., Rossi, R.A., Wang, H., McAuley, J.: Knowledge-Aware Query Expansion with Large Language Models for Textual and Relational Retrieval. In: Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics (2025)
22. Yang, H., Zhang, Q., Jiang, W., Li, J.: Pasemiqa: Plan-assisted agent for question answering on semi-structured data with text and relational information. arXiv preprint arXiv:2502.21087 (2025)
23. Lee, M.C., et al.: Hybgrag: Hybrid retrieval-augmented generation on textual and relational knowledge bases. arXiv preprint arXiv:2412.16311 (2024)
24. Li, Y., Zhang, R., Liu, J.: An enhanced prompt-based llm reasoning scheme via knowledge graph-integrated collaboration. In: International Conference on Artificial Neural Networks, pp. 251–265 (2024)
25. Lei, Y., et al.: Mixture of Structural-and-Textual Retrieval over Text-rich Graph Knowledge Bases. In: Findings of the Association for Computational Linguistics: ACL 2025, pp. 18306–18321 (2025)
26. Clemedtson, Alfred, and Borun Shi. "GraphRAFT: Retrieval Augmented Fine-Tuning for Knowledge Graphs on Graph Databases." arXiv preprint arXiv:2504.05478 (2025)
27. Boer, D., Roth, S., Kramer, S.: Focus, Merge, Rank: Improved Question Answering Based on Semi-structured Knowledge Bases. arXiv preprint arXiv:2505.09246 (2025)
28. Robertson, S.E., Zaragoza, H.: The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval (2009)
29. Izacard, Gautier, et al. "Unsupervised dense information retrieval with contrastive learning." arXiv preprint arXiv:2112.09118 (2021).
30. Xiong, L., et al.: Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval. In: ICLR (2021)
31. OpenAI. 2023. OpenAI Embeddings API. [Software].
32. Voyage AI. 2023. Voyage AI Embeddings API. [Software]
33. Parishad BehnamGhader, Vaibhav Adlakha, Marius Mosbach, Dzmitry Bahdanau, Nicolas Chapados, and Siva Reddy. 2024. LLM2Vec: Large Language Models Are Secretly Powerful Text Encoders. arXiv preprint (2024)
34. Niklas Muennighoff, Hongjin Su, Liang Wang, Nan Yang, Furu Wei, Tao Yu, Amanpreet Singh, and Douwe Kiela. 2024. Generative Representational Instruction Tuning. arXiv:2402.09906 [cs.CL]
35. Yasunaga, M., et al.: QA-GNN: Reasoning with Language Models and Knowledge Graphs for Question Answering (2021)
36. Sun, J., et al.: Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. In: The Twelfth International Conference on Learning Representations (2024)
37. Luo, L., Li, Y.F., Haf, R., Pan, S.: Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning. In: The Twelfth International Conference on Learning Representations (2024)
38. Wu, S., et al.: Stark: Benchmarking llm retrieval on textual and relational knowledge bases. In: NeurIPS Datasets and Benchmarks Track (2024)
39. Wu, S., et al.: Avatar: Optimizing llm agents for tool usage via contrastive reasoning. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 37, pp. 25981–26010 (2024)

40. Xia, Yu, et al. "Knowledge-aware query expansion with large language models for textual and relational retrieval." Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers). 2025.